Tracking and Grasping Color Blocks

Tracking and Grasping Color Blocks

1. Course Content
2. Preparation
3. Case Demonstration
4. Color Block Color Calibration
5. Core Source Code Analysis

1. Course Content

The feature implements the program acquiring images through the camera and, by key press, selecting the color block to be tracked and grasped. The program will recognize the target color block in the image. When the color block is moved by hand, the robotic arm follows along. When the robotic arm aligns with the target, the chassis moves into the grasping range, and then the robotic arm goes to grasp the color block and places it in a set position.

> **Important**
>
> The grasping point calculation methods differ between the depth camera and the USB monocular camera: the depth camera obtains the spatial coordinates of the target center point through the depth map, while the monocular camera performs PnP solving to obtain the spatial position of the target center point using a known side length (0.04m)

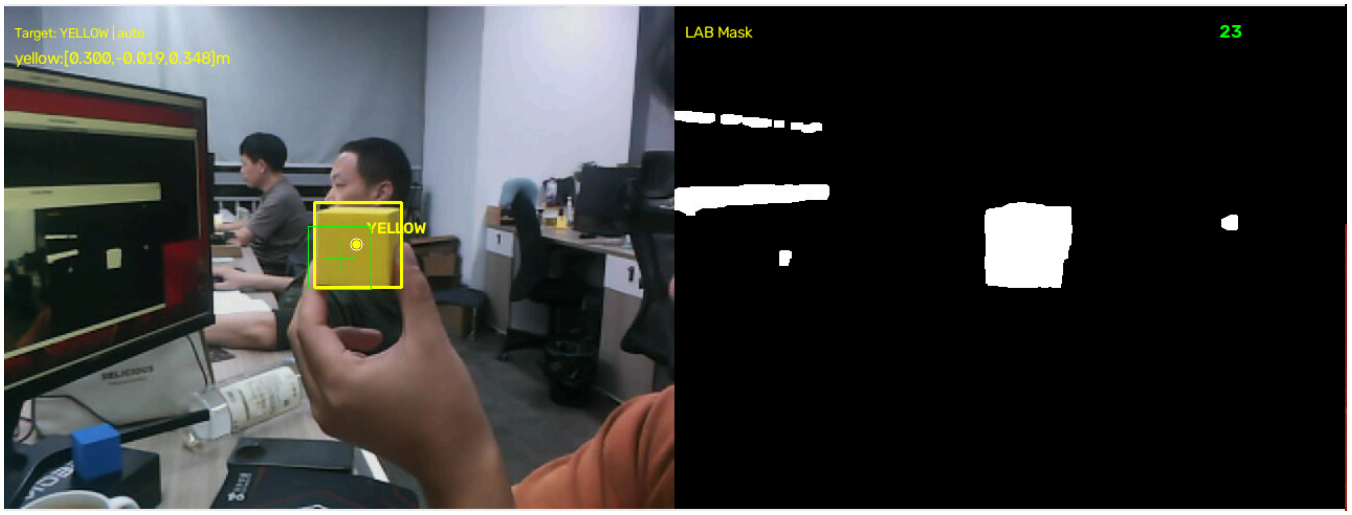
2. Preparation

The wooden block used in this section: **40x40x40mm color block**.

```
ros2 launch yahboomcar_bringup arm_demo.launch.py demo:=color_follow_grasp
```

3. Case Demonstration

- Here, tracking and grasping the yellow block is taken as an example; press Y to detect yellow. As shown in the figure below, after the program runs, hold the **40x40x40mm** yellow block provided with the product so that it appears in the image
- The robotic arm will first track the target so that the camera crosshair is directly aimed at the color block
- After the color block's center stably enters the central target box (the crosshair turns green), the robotic arm camera pose is locked, and the chassis automatically drives smoothly forward toward the color block and performs grasping



4. Color Block Color Calibration

Refer to the color calibration steps in the previous section's tutorial 【Color Block Color Sorting】; the calibration method is the same.

5. Core Source Code Analysis

- Program path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/arm_demo/color_follow_grasp.py
```

_grasp_worker function (grasping worker thread):

This function is the main control thread of the grasping sequence, sequentially completing obtaining the color block pose, angle calculation, inverse kinematics solution, robotic arm motion, gripper closing, placement, and other steps.

- `time.sleep(0.5)`: waits for the camera image to stabilize before starting to grasp
- `color_name = self._target_color`: obtains the currently selected target color; returns an error if not set
- `self._latest_blocks`: traverses the latest detected color block list, looking for a target whose color matches and that has a measure pose
- `b['color'] == color_name`: filters target color blocks by color
- `b.get('measure_pose')`: obtains the 3D pose of the color block in the measure coordinate frame; skips if there is no pose
- `_compute_block_grasp_angle(block['rect'])`: calculates a signed grasp rotation angle (-45°~45°) from the minimum enclosing rectangle of the color block, used to determine the gripper rotation direction, consistent with the rvec yaw convention of AprilTag grasping
- `grasp_offset`: grasping offset `[gx, gy, gz, pitch]`, compensating for the difference between the

camera-detected position and the actual grasping position

- `arm_kinematics.inverse_kinematics(...)`: calls the inverse kinematics solver, taking the target position (camera pose + grasping offset) and pitch angle as input, and outputting 5 joint angles
- `result.approximate` / `result.adjusted_pitch`: handles the cases of approximate solutions and automatic pitch angle adjustment
- `joints[0:3] = round(ik_joints[0:3])`: rounds the J1~J4 joint angles obtained from the inverse solution to integers (the low-level firmware only accepts integers)
- `joints[4] = round(90 + angle_norm)`: the J5 joint angle is adjusted according to the calculated rotation angle, so that the gripper rotation aligns with the color block direction
- `joints[5] = 30`: the J6 gripper angle is set to 30° (open state)
- `pubSix_Arm(joints, runtime=1500)`: publishes the 6-joint control command, and moves to the grasping pose
- `SingleJoint_pub.publish(ArmJoint(id=6, joint=gripping_angle, time=1000))`: controls the gripper to close individually
- According to `color_name` (red/green/blue/yellow), selects a different placement joint angle `down_joint`, and places the color block at the placement position of the corresponding color
- `SingleJoint_pub.publish(ArmJoint(id=6, joint=30, time=1500))`: opens the gripper to release the object
- `pubSix_Arm(observation_pose)`: returns the robotic arm to the observation pose
- `finally: self._reset_grasp_state()`: regardless of success or failure, resets the grasping state at the end

```
    def _grasp_worker(self):
        """Worker thread: execute the grasp sequence (IK -> move to target -> close gripper
-> place -> return to observation pose)"""
        try:
            time.sleep(0.5)  # wait for the camera to stabilize

            # Get the latest block data
            color_name = self._target_color
            if color_name is None:
                self.get_logger().error("No target color set for grasp")
                return

            block = None
            pose = None
            for b in self._latest_blocks:
                if b['color'] == color_name:
                    p = b.get('measure_pose')
                    if p is not None:
                        block = b
                        pose = p
                        break

            if block is None or pose is None:
                self.get_logger().error("Color block lost before grasp")
```

```
                return

            # Compute the signed grasp rotation angle (-45~45 degrees; direction matches the
apriltag grasp rvec yaw convention)
            angle_norm = self._compute_block_grasp_angle(block['rect'])

            # Apply grasp_offset to the IK target position
            mx, my, mz = pose
            gx, gy, gz = self.grasp_offset[0], self.grasp_offset[1], self.grasp_offset[2]
            self.get_logger().info(
                f"measure pose: x={mx:.3f}, y={my:.3f}, z={mz:.3f}; "
                f"IK target: x={mx+gx:.3f}, y={my+gy:.3f}, z={mz+gz:.3f}, pitch=
{self.grasp_offset[3]:.1f}°"
            )

            result: KinResult = self.arm_kinematics.inverse_kinematics(
                target_x=mx + gx,
                target_y=my + gy,
                target_z=mz + gz,
                target_pitch=self.grasp_offset[3],
                seed_angles_deg=None,
            )

            if not result.success:
                self.get_logger().error(f"Grasp IK failed: {result.message}")
                return

            if result.approximate:
                self.get_logger().warn(f"IK approximate solution, error=
{result.position_error:.4f}m")
            if result.adjusted_pitch is not None:
                self.get_logger().warn(f"IK pitch sweep, adjusted pitch=
{result.adjusted_pitch:.1f}°")

            # Build the joint angles
            ik_joints = result.data
            joints = [0, 0, 0, 0, 0, 0]
            joints[0] = round(ik_joints[0])
            joints[1] = round(ik_joints[1])
            joints[2] = round(ik_joints[2])
            joints[3] = round(ik_joints[3])
            joints[4] = round(90 + angle_norm)
            joints[5] = 30

            self.get_logger().info(f"Grasp joints: {joints}")
            self.pubSix_Arm(joints, runtime=1500)
            time.sleep(3.0)
            self.SingleJoint_pub.publish(ArmJoint(id=6, joint=self.gripping_angle,
time=1000))
            time.sleep(3.0)

            # Move to the color-specific release position
            if color_name == 'red':
```

```
                down_joint = [180, 35, 70, 5, 60, self.gripping_angle]
            elif color_name == 'green':
                down_joint = [180, 35, 60, 0, 90, self.gripping_angle]
            elif color_name == 'blue':
                down_joint = [0, 30, 70, 2, 90, self.gripping_angle]
            elif color_name == 'yellow':
                down_joint = [0, 43, 48, 6, 90, self.gripping_angle]
            else:
                down_joint = [180, 35, 70, 5, 60, self.gripping_angle]
            self.pubSix_Arm(down_joint)
            time.sleep(2.5)

            # Release the gripper
            self.SingleJoint_pub.publish(ArmJoint(id=6, joint=30, time=1500))
            time.sleep(2.0)

            self.pubSix_Arm(list(self.observation_pose))
            time.sleep(2.5)

            self.get_logger().info("Grasp sequence complete")
        finally:
            self._reset_grasp_state()
```

- Parameter file

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/common.yaml
```

Parameter Description:

- `observation_pose: [90, 150, 12, 20, 90, 30]`: the 6 joint angles (degrees) of the observation pose, at which the robotic arm has the best field of view
- `grasp_offset: [0.0, 0.0, 0.02, 0.0]`: grasping offset `[x, y, z, pitch]`, compensating for the difference between the camera-detected position and the actual grasping position; this parameter can be adjusted when there is a grasping deviation
- `gripping_angle: 130`: the angle (degrees) when the gripper is closed; the larger the value, the stronger the gripping force
- `track_distance_threshold: 0.2`: tracking distance threshold (meters); when the color block distance exceeds this value, the chassis automatically advances toward the target
- `track_pixel_tolerance: 30`: pixel alignment tolerance; alignment is considered achieved when the deviation between the color block center and the image center is less than this value
- `gaze_step: 3`: the step (degrees) of each joint adjustment; the larger the value, the faster the tracking response but the more likely overshoot
- `ema_alpha: 0.3`: EMA filter coefficient (0~1); the smaller the value, the smoother the filtering but the slower the response
- `alignment_stable_count: 3`: consecutive alignment count threshold; tracking is considered stable only after reaching this count, which can then trigger grasping

- `min_block_area: 300`: minimum color block area (pixels), filtering out noise detection results with too small an area
- `square_size: 0.04`: the physical side length (meters) of the block in USB monocular mode, used for PnP solving of the target's 3D position
- `usb_camera_measure: [0.012, 0.0, 0.013]`: the pose compensation value (meters) in USB monocular mode, correcting the deviation between the camera mounting position and the origin of the measure coordinate frame

```
color_follow_grasp:
  ros__parameters:
    observation_pose: [90, 150, 12, 20, 90, 30]
    grasp_offset: [0.0, 0.0, 0.02, 0.0]    # x,y,z axis offset distance and included angle
    gripping_angle: 130                     # gripper grasping angle, in degrees, integer
    track_distance_threshold: 0.2          # tracking distance threshold (meters); the
chassis moves when exceeded
    track_pixel_tolerance: 30               # pixel alignment tolerance
    gaze_step: 3                            # joint adjustment step per cycle (degrees)
    ema_alpha: 0.3                          # EMA filter coefficient (0~1), the smaller the
smoother
    alignment_stable_count: 3               # consecutive alignment count threshold;
considered stable only after reaching it
    min_block_area: 300                     # minimum color block area (pixels), filtering
noise
    square_size: 0.04                       # block side length in usb monocular mode
(meters), default 4x4cm
    usb_camera_measure: [0.012, 0.0, 0.013] # pose compensation in usb monocular mode
(meters)
```